

RepoHIVE: Hierarchical Codebase Indexing Framework for Large-Scale Repositories and Agentic Software Engineering

Abstract

Large software repositories present a scale problem for both human developers and AI coding agents: a real project can contain thousands of files and tens of thousands of dependency relationships, forming a flat graph that is too large to visualize, too large to fit in a language model's context window, and poorly served by the directory tree, which reflects how files were stored rather than how code depends on itself. Existing tools either build a flat dependency graph and retrieve from it, or apply a single global clustering strategy to the entire codebase, or generate hierarchies from language-model summaries that differ from run to run. We present RepoHIVE (Repository Hierarchical Indexing and Visualization Engine), a system that transforms a flat dependency graph extracted from a Java repository into a navigable, multi-level hierarchy (Repository $\rightarrow$ Groups $\rightarrow$ Files $\rightarrow$ Functions). The central contribution is adaptive, per-region hierarchy construction driven by structural quality: rather than imposing one global grouping method, RepoHIVE measures the cohesion and coupling of each region and decides, region by region, whether to preserve the existing package or directory boundary (when the region is already well-structured) or to reconstruct it via dependency-based community detection (when it is poorly-structured). Every score, decision, and parameter is recorded so that the output is fully deterministic, reproducible, and auditable. The resulting JSON index acts as a single stable contract consumed by three clients: a semantic-zoom web viewer for humans, a bounded-context traversal model for AI agents, and a static blast-radius query for impact analysis. We describe the architecture, the algorithm, the determinism strategy, and an evaluation design framed strictly around code navigation. We report the working parsing layer as preliminary validation and state the project's limitations openly. All performance figures in this manuscript are engineering estimates unless explicitly marked as measured.

Index Terms - software comprehension, dependency graph, hierarchical clustering, community detection, code navigation, static analysis, determinism, program understanding, retrieval for code, software architecture recovery.

I. Introduction

Software systems grow, and as they grow they become harder to understand. A mature enterprise codebase routinely contains thousands of source files, tens of thousands of classes

and functions, and a dense web of dependencies between them - one class imported in dozens of places, one utility called across the whole system. Two very different consumers now need to make sense of this scale. The first is the human developer, who may be onboarding onto an unfamiliar project and must build a mental model before making any change; empirical studies report that program comprehension consumes more than half of a developer's working time [3]. The second is the AI coding agent, which is increasingly used to complete, repair, and refactor code but whose effective working memory - the language-model context window - holds only a few thousand tokens, far less than a single large module.

The natural artifact for reasoning about a codebase at this scale is the dependency graph: a directed graph whose nodes are code entities (files, classes, functions) and whose edges are the imports and calls between them. Unfortunately, at real scale the dependency graph is not a solution but a restatement of the problem. A graph of thousands of nodes and tens of thousands of edges is an unreadable tangle: it cannot be rendered on a screen in a way a human can follow, it cannot be handed wholesale to a language model, and it offers no obvious entry point for exploration. Figure 1 illustrates this contrast - the flat graph on the left versus the navigable hierarchy that RepoHIVE produces on the right.

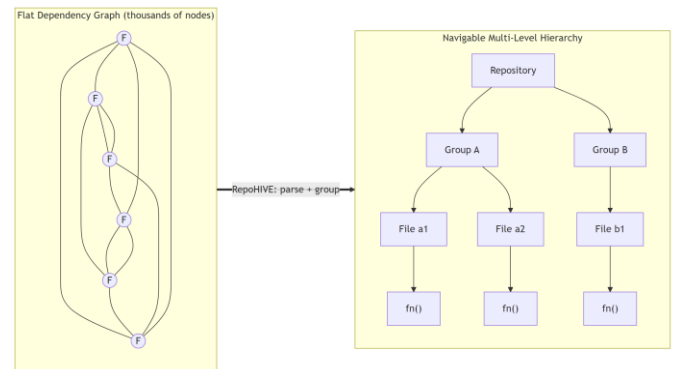


Figure 1. The core problem and the RepoHIVE transformation: a flat dependency graph (unreadable at scale) is transformed into a navigable multi-level hierarchy.

The reflexive answer - "just use the folder structure" - fails on inspection. Directory trees record how files were saved, not how the code actually depends on itself. A well-named folder may contain classes that barely interact, while two tightly coupled classes may live in different directories. Folder structure is therefore a weak and sometimes misleading proxy for the structure that matters for comprehension and navigation.

RepoHIVE addresses this by constructing, from the dependency graph, a multi-level hierarchy of the form Repository $\rightarrow$ Level 1 Groups $\rightarrow$ Level 2 Groups $\rightarrow$ Files $\rightarrow$ Functions, so that a user or an agent can start at the top, see a

small number of large regions, descend into the one that matters, and never confront the full graph at once. The defining research contribution is that this hierarchy is built adaptively: RepoHIVE measures the structural quality of each region and decides per region whether to preserve its existing boundary or to rebuild it from the dependency structure, rather than applying one global strategy everywhere. Every decision is recorded, and the whole pipeline is deterministic.

The contributions of this paper are: (1) Adaptive per-region construction - a hierarchy-construction policy that assesses each region's cohesion and coupling and chooses between a preserve action (keep the existing package/directory boundary) and a reconstruct action (rebuild via community detection), instead of a single global clustering method; to our knowledge this per-region preserve-versus-reconstruct decision, driven by structural quality, is not present in the existing software-clustering literature. (2) Determinism and auditability as first-class properties - the same repository always produces the same hierarchy: identifiers are content-derived, iteration is over canonically sorted collections, and community detection runs with a fixed seed; every region's scores, decision, decision confidence, and the calibrated boundary are written to metadata, so the boundary is an auditable, tunable parameter rather than a hidden magic number. (3) A stable JSON contract with three consumers - one index format serves a semantic-zoom human viewer, a bounded-context traversal model for AI agents (a direct response to the "lost in the middle" degradation of language models on long contexts [8]), and a static blast-radius impact query. (4) A working, deterministic parsing layer that produces the contract from real Java repositories, reported here as preliminary validation.

II. Related Work

Flat code graphs and retrieval. A line of recent systems builds a dependency or call graph of a repository and retrieves from it to support an AI task. RepoGraph [7] constructs a line-level code graph with Tree-sitter and supplies ego-graph context to coding agents, improving bug-fix resolve rates on real GitHub issues. Knowledge-graph approaches to repository-level code generation [2] store code entities and relations in a graph database and retrieve relevant sub-graphs before generation. RepoCoder [9] iterates retrieval and generation using similarity search over sliding-window code snippets. These systems demonstrate convincingly that structural, cross-file context improves AI performance on code tasks, and they share our reliance on AST-based graph extraction. They differ from RepoHIVE in that they operate over a flat graph: they retrieve well when the target is known, but they offer no multi-level organization for exploration and no notion of grouping quality.

Hierarchical summarization. Two recent systems build hierarchies over code. Code-Craft [6] performs bottom-up

call-graph summarization, propagating summaries upward, and reports large retrieval gains on big codebases; a hierarchical summarization approach for business applications [3] segments files via the AST and aggregates function summaries into file- and package-level summaries. Both confirm that hierarchy helps at scale, but both derive the hierarchy from either call-graph depth or on-disk file structure, and both rely on language-model-generated summaries, making their hierarchies non-deterministic. RepoHIVE instead derives grouping from measured structural quality of the cross-file dependency graph and is fully deterministic.

Software module clustering. A large body of work clusters software modules to recover architecture or aid comprehension. The systematic review by Sarhan et al. [10] catalogs 143 such papers and identifies dependency-graph-based clustering with community-detection or agglomerative algorithms as the state of the art, with Modularization Quality (MQ) and MoJoFM as standard evaluation metrics; it explicitly names adaptive clustering that adjusts to system characteristics as an open direction. Stana and Şora [5] combine structural and co-change ("logical") dependencies and evaluate Louvain, Leiden, and DBSCAN, confirming both the value of richer edge signal and the effectiveness of Louvain-family detectors on Java systems. Crucially, every clustering approach in this literature applies a single global strategy to the whole codebase; none measures quality per region and switches strategy accordingly.

Software architecture quality. Druskat et al. [4] argue that architectural quality - measured through cohesion, coupling, cyclomatic complexity, and change-coupling - governs the maintainability of software, and that high-level architectural problems are poorly captured by ordinary code-smell detectors. This provides the theoretical grounding for the cohesion and coupling measures RepoHIVE uses to drive its preserve-versus-reconstruct decision.

Context injection for language models. Ehsani et al. [1] show that structured, layered injection of repository and project context substantially improves LLM program repair, while injecting all context at once degrades it. Liu et al. [8] rigorously demonstrate the "lost in the middle" effect: language models use information at the start and end of a long context far better than information in the middle, and this holds even for very long context windows. These two results jointly motivate RepoHIVE's design goal of delivering a small, relevant branch of the hierarchy to an agent rather than the whole repository.

Positioning. RepoHIVE sits at the intersection of these lines: it borrows AST-based graph extraction from the code-graph systems, the cohesion/coupling quality lens from the architecture literature, and Louvain community detection from the clustering literature - but combines them in a way none of

them does, via a deterministic, per-region, quality-driven preserve-versus-reconstruct policy that yields a multi-level, auditable hierarchy. We make the narrow claim that this specific combination is novel; we do not claim to outperform mature commercial code-intelligence platforms on their broad feature set.

III. Problem Formulation

Let a repository be represented by a directed graph $G = (V, E)$, where each node $v \in V$ is a code entity with a kind (file, class, or function) and metadata including its declared package path and its directory path, and each edge $e \in E$ is a dependency from a source entity to a target entity carrying three non-negative signals: import frequency, method-call frequency, and shared-type count. This is the flat dependency graph produced by parsing the repository.

The navigation problem is to construct a hierarchy H over the leaf entities of G such that: H has a single root (the repository) and a bounded branching factor at every internal node, so no single view forces the user or agent to consider more than a small number of children at once; every file is reachable by exactly one root-to-leaf path, so locating any file requires a bounded number of expansions; every original dependency in E is preserved, and dependencies that cross group boundaries are aggregated and represented at the level where the boundary is crossed, so navigating along a dependency never loses information; and the construction is deterministic, so the same G and configuration always yield the same H .

The research question layered on top of this is how to form the groups. A pure-preserve strategy keeps the existing package/directory boundaries; it is faithful to author intent where the authors organized the code well, but it propagates disorder where they did not. A pure-reconstruct strategy rebuilds every boundary from the dependency structure via community detection; it can rescue a tangled region, but it needlessly discards good existing structure and can fragment a well-designed package. Neither global strategy is right everywhere, because real repositories are not uniformly structured - some packages are clean and some are messy.

RepoHIVE's thesis is therefore that construction should be adaptive per region: measure how well-structured each region already is, and preserve it if it is good, reconstruct it if it is poor. This yields two separable claims, which we keep distinct to avoid overreach. Claim A (hierarchy beats flat, for navigation and retrieval): because human and agent attention are bounded, a tree that shows a handful of items per level is easier to navigate and cheaper to feed to a context-limited model than a flat graph or a flat pile of clusters; this is a robust, easily demonstrated claim about navigation, not a claim of more accurate clustering. Claim B (adaptive beats single-global, on mixed-quality repositories): on repositories whose regions vary in structural quality, an adaptive policy

should never be worse than the better of the two global baselines and should be strictly better where quality is mixed; this is stated as a hypothesis to be evaluated, not a theorem.

The evaluation (Section X) is framed strictly around code navigation and explicitly not around recovering a "ground-truth" architecture; we make no claim that reconstructed groups correspond to intended architectural modules.

IV. System Architecture

RepoHIVE is organized as a stateless, file-handoff pipeline of three stages - parse, group, and view - connected by JSON artifacts on disk (Figure 2). Each stage is a batch process that reads its input, writes its output, and exits; there is no always-on backend in the core engine. This design keeps the stages independently testable and makes every intermediate artifact inspectable.

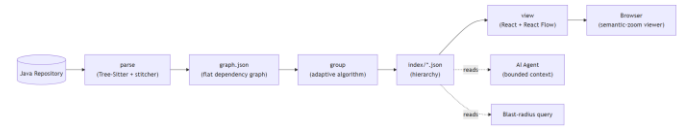


Figure 2. The RepoHIVE stateless file-handoff pipeline. *parse* turns a Java repository into *graph.json*; *group* turns that into the *index/ hierarchy*; *view* renders it. The index is also read directly by AI agents and by blast-radius queries.

Parse. The parsing layer uses Tree-Sitter to produce a per-file abstract syntax tree (AST) for each Java source file, then stitches those per-file ASTs into a single cross-file dependency graph. A key architectural fact is that Tree-Sitter alone yields only per-file syntax trees; the cross-file dependency graph - the artifact that matters - is produced by RepoHIVE's own stitching logic, which resolves imports and references across files. The ASTs are transient (one file in memory at a time, then discarded); the persisted artifact is *graph.json*.

Group. The grouping layer is the research core. It ingests *graph.json*, weights the edges, assesses each region's structural quality, adaptively preserves or reconstructs each region, assembles the multi-level hierarchy, and serializes a five-file index set. Section V details this stage.

View. The viewer is a React and React Flow web application that renders the hierarchy with semantic zoom: only about one level (a handful of nodes) is drawn at a time, so rendering cost is bounded regardless of repository size. A flat-graph baseline viewer is retained deliberately, so the difference between an unusable flat layout and a navigable hierarchical one can be shown side by side.

The JSON contract as the universal seam. The stages never call each other directly; they communicate only through JSON files whose shape is fixed. *graph.json* is written by the parser

and read by the grouping layer; the index/ set is written by the grouping layer and read by every downstream consumer. Because the contract is stable, new consumers (an AI-agent adapter, a query server, an alternative viewer) attach without any change to the engine. Engine versus ecosystem: the pipeline above is the engine - purely local, no cloud, no authentication, no database. Distribution wrappers such as a CLI, an editor plugin, or a query server are ecosystem concerns that layer outside the engine and consume the same JSON contract; they are out of scope for this paper.

V. The Adaptive Grouping Algorithm

The grouping layer is a pipeline of pure, independently testable stages, each consuming an immutable input and

producing a new immutable value.

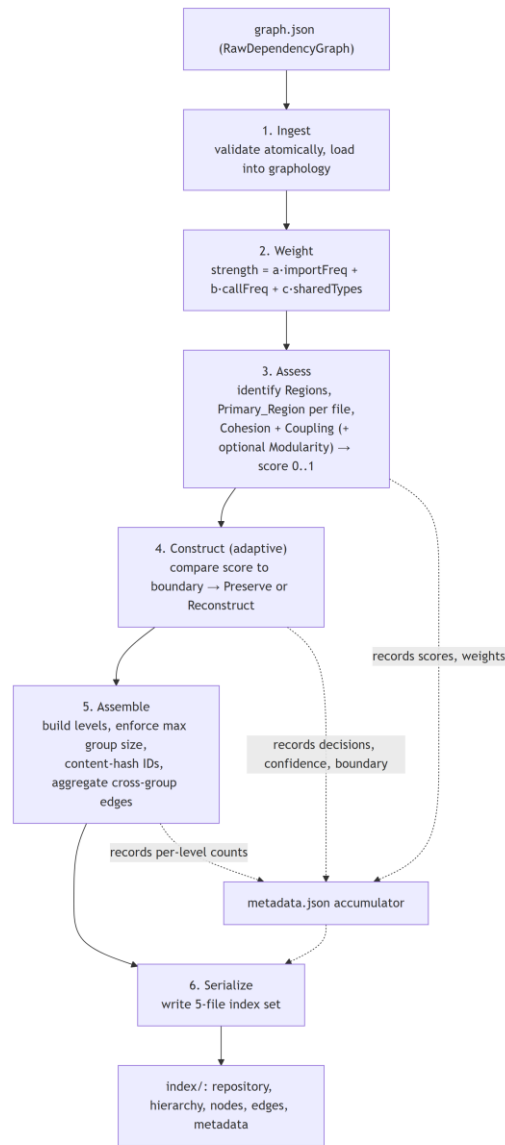


Figure 3. The six-stage grouping algorithm: ingest, weight, assess, construct, assemble, serialize, with a metadata accumulator threaded through the quality-bearing stages.

A. Ingest

The ingestor validates the input atomically and loads it into an in-memory directed weighted graph. Validation is all-or-nothing and ordered: absent/null input, then zero nodes, then duplicate node identifiers, then edges referencing a missing node. On any defect it returns a typed error and performs no partial load; on success the in-memory model contains exactly the input node and edge sets, with no additions or removals.

B. Weight

Each edge is assigned a single non-negative dependency strength combining its three signals. The default is a linear combination, $\text{strength} = a \cdot \text{importFreq} + b \cdot \text{callFreq} + c \cdot \text{sharedTypes}$, with non-negative coefficients; an all-zero-signal edge receives strength zero. The weighting is deterministic and leans monotonic (more signal never decreases strength under the default function) without being required to be strictly monotonic, leaving room to refine the function later without breaking the contract.

C. Assess

The assessor identifies candidate regions and assigns every file to exactly one primary region by a deterministic precedence rule: the most-specific declared package (or module) boundary containing the file, or, if none is declared, the most-specific directory subtree. In the Phase-1 Java implementation each declared package is a region, so region ownership over files is total and non-overlapping. For each region it computes: Cohesion - the summed strength of edges internal to the region, normalized by the region's node count (a higher value means the region's members are more strongly related to one another); Coupling - the summed strength of edges crossing the region boundary, normalized by the total strength incident to the region (a lower value means better isolation; by construction this is a ratio in $[0, 1]$); and, optionally, Modularity - Newman's Q for the partition induced by the region boundaries, deliberately not the primary discriminator, because Q is exactly what the reconstruct step's community detection optimizes, so using it to decide whether to run community detection would beg the question.

Because the three raw metrics live on incompatible scales (cohesion is unbounded above, coupling is already a ratio, modularity is roughly $[-0.5, 1]$), each is mapped into $[0, 1]$ before being combined. Cohesion is squashed by $\text{cohesion} / (\text{cohesion} + k)$ for a documented constant k (default 1.0) that fixes the neutral midpoint; coupling contributes its complement $(1 - \text{coupling})$ since lower coupling is better; modularity, when used, is mapped affinely from its theoretical range. The Structural Quality Score is the weighted sum of these normalized terms with non-negative weights that sum to one, so the score is guaranteed to lie in $[0, 1]$. A region with fewer than two nodes or no internal edges bypasses normalization and receives a documented degenerate score rather than an undefined value. Crucially, each normalization

depends only on the region's own metric and documented constants - never on the spread of other regions in the run - so a region's score does not change when unrelated regions are added or removed. All weights and constants are written to metadata.

D. Construct (adaptive)

This is the research core. For each primary region the constructor compares the region's score to a configurable Structural Quality Boundary and chooses exactly one action (Figure 4): Preserve ($\text{score} \geq \text{boundary}$) - the region is already well-structured, so its existing package/directory boundaries are kept as group boundaries; Reconstruct ($\text{score} < \text{boundary}$) - the region is poorly-structured, so its groups are rebuilt by running community detection over the region's nodes and strength-weighted edges.

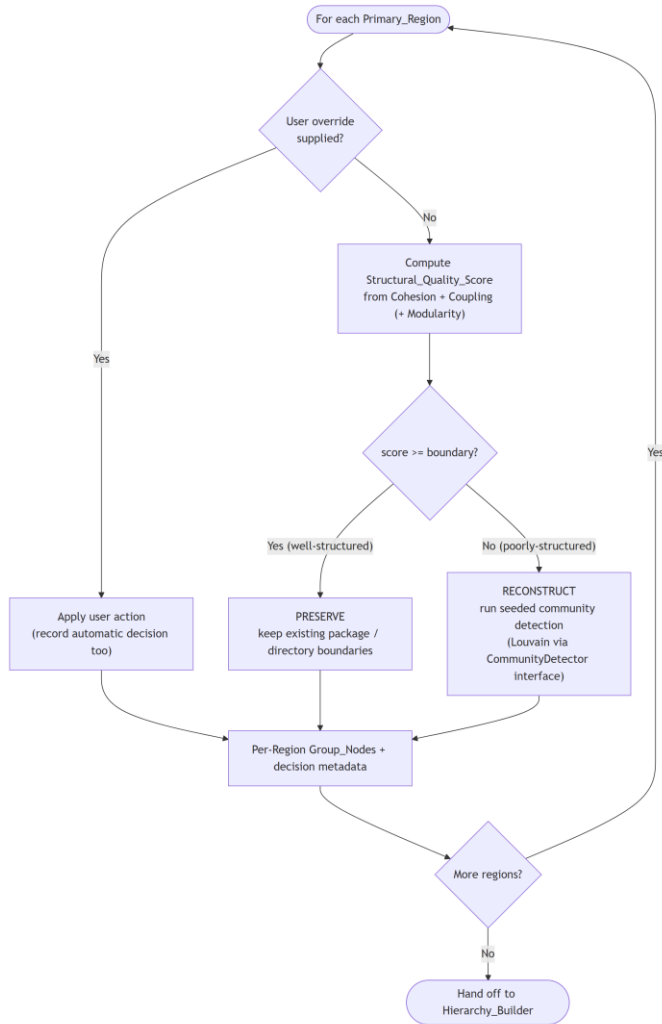


Figure 4. The per-region preserve-versus-reconstruct decision.

A user may override the automatic decision for any region; when this happens both the applied action and the automatically computed decision are recorded. For every

region the constructor records its cohesion, coupling, (optional) modularity, combined score, the applied action, the automatic action, whether it was overridden, and the decision confidence (the absolute distance between the score and the boundary), which highlights regions that flip near the boundary.

Community detection behind an interface. The reconstruct action does not depend on Louvain directly; it depends on a CommunityDetector interface. This is deliberate: the research contribution is the adaptive construction policy, not any single detector. Phase 1 ships a Louvain-based implementation; Leiden, Infomap, or label-propagation detectors can be substituted without changing any other component - which also makes a Louvain-versus-Leiden comparison a low-cost future experiment. Because the default Louvain implementation is order- and seed-sensitive, our implementation runs it with a fixed seeded pseudo-random generator over canonically sorted nodes and edges, then relabels the resulting communities by a content-derived key, so the reconstruct action is fully reproducible.

E. Assemble

The hierarchy builder assembles the per-region group results into the multi-level tree of Figure 5: a single repository root at level 0, then Level 1 and Level 2 groups, then files, then the functions defined in each file. It enforces a configurable maximum group size (default 20, range 2–50): any group exceeding the limit is partitioned into the provably minimum number of balanced subgroups - $\text{ceil}(n / \text{maxGroupSize})$ subgroups whose sizes differ by at most one, filled from a canonically sorted child list. This bounds the branching factor at every node so no single view overwhelms the reader, and derives hierarchy depth from the file count and the size bound rather than fixing it in advance.

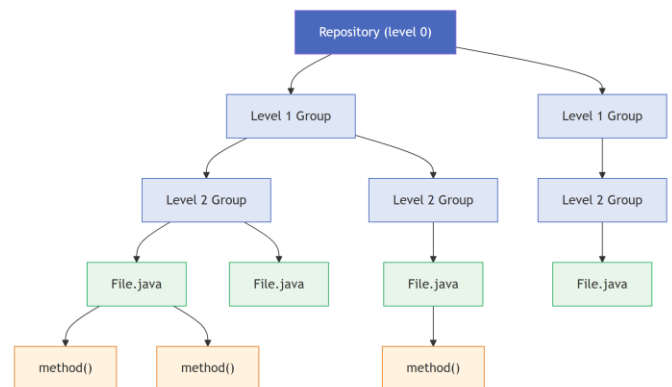


Figure 5. The four-level hierarchy produced by RepoHIVE: Repository → Level 1 Groups → Level 2 Groups → Files → Functions.

The builder preserves all dependency information. Every original leaf-to-leaf edge is retained with its direction and

strength. When two connected leaves fall under different groups, a cross-group edge is created between their immediate parent groups at the lowest level where their ancestors differ, and its weight is the sum of the strengths of the underlying leaf edges it aggregates. The set of leaf-to-leaf reachability relationships in the output equals that of the input, so no dependency is lost or invented by the grouping.

F. Serialize

Finally the hierarchy and its metadata are written as a five-file index set - repository.json, hierarchy.json, nodes.json, edges.json, and metadata.json - which can be read back with full round-trip fidelity (same node set, edge set, per-region decisions, and depth). The metadata file additionally records per-level node and edge counts, total cross-group edges, and the average branching factor, so the scalability of the hierarchical output can be quantified against the flat graph directly from the artifact.

VI. Determinism and Reproducibility

Determinism is a hard requirement, not a nicety: an evaluation that compares construction strategies, or a sensitivity analysis that sweeps the decision boundary, is only meaningful if re-running the pipeline on the same input reproduces the same hierarchy exactly. RepoHIVE enforces determinism with three rules applied throughout the pipeline. First, canonical ordering: any iteration over nodes or edges that can affect output is performed over a collection sorted by node identifier (and, for edges, by the source/target pair), never over insertion or input order; consequently, permuting the order of nodes or edges in the input leaves the output hierarchy unchanged. Second, no ambient nondeterminism: identifiers are never derived from counters, timestamps, wall-clock values, memory addresses, random values, or input position, and community detection uses a fixed seed. Third, content-addressed identifiers: every group node's identifier is a hash of its canonicalized membership, so identical contents always yield the identical identifier regardless of how the group was produced, and distinct memberships yield distinct identifiers.

Together these rules make the entire transformation a deterministic function of the input graph and the configuration. This is what turns the Structural Quality Boundary from a hidden constant into an auditable, tunable research parameter: because each region's recorded score deterministically reproduces its preserve/reconstruct decision at any boundary value, a boundary sweep becomes a pure post-processing exercise over the recorded metadata, with no need to re-run assessment. The correctness of these properties is checked by property-based testing: rather than a handful of hand-written examples, generators produce large families of valid dependency graphs - with controllable size, package structure, diamond and cyclic dependency shapes, and degenerate single-node or edgeless regions - and each invariant (set-preserving ingestion, score range, partition

minimality, round-trip fidelity, order-independence, cycle-safe blast radius) is asserted across many generated inputs.

VII. The JSON Contract and Its Consumers

The stable JSON contract is what lets one engine serve three very different consumers without special-casing any of them. Figure 6 shows the core data model.

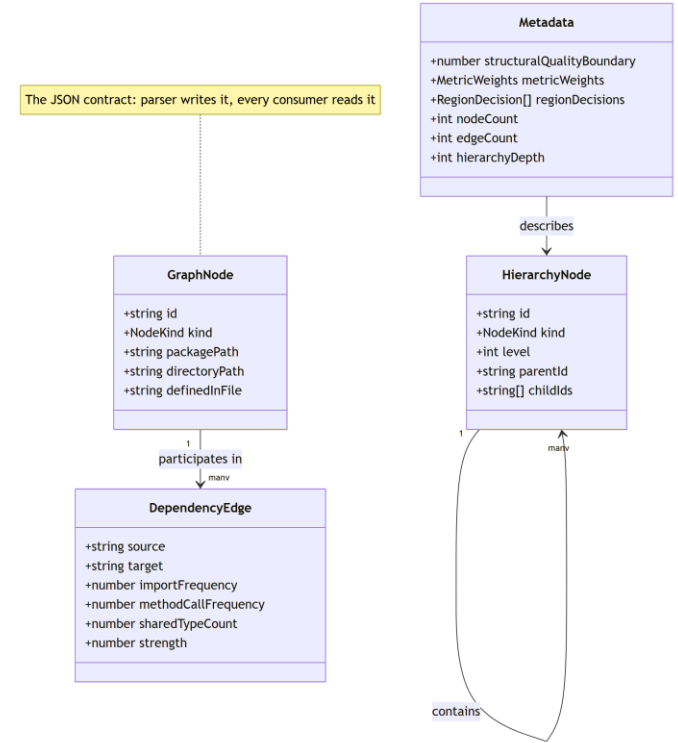


Figure 6. The JSON contract data model: *GraphNode* and *DependencyEdge* (the parser's output), *HierarchyNode* (the grouping layer's output), and *Metadata* (the audit record). The parser writes it; every consumer reads it.

Consumer 1 - the human viewer. The React/React Flow application reads the hierarchy and renders it with semantic zoom, showing roughly one level at a time. This directly supports Claim A: bounded cognition is respected because the viewer never draws the whole graph. Consumer 2 - the AI agent. An agent reads the same index and descends the hierarchy to load only the relevant branch as context, instead of the whole repository. This is a direct engineering response to the "lost in the middle" finding [8]: because a language model uses the middle of a long context poorly, the right move is not a bigger prompt but a smaller, better-targeted one. The hierarchy provides the structure needed to select that small, relevant slice - and the layered-context result of Ehsani et al. [1] suggests such structured delivery outperforms dumping everything at once. Consumer 3 - blast-radius queries. The same index answers impact questions statically, as described next. Because all three consume the identical contract, the

engine is decoupled from every one of them; adding a fourth consumer requires no engine change.

VIII. Blast-Radius Analysis

Given a modified entity, a developer often wants to know what else could be affected. RepoHIVE answers this with a blast-radius query over the preserved dependency edges: starting from the target node, it traverses edges in the dependent-to-dependency direction (following who depends on whom) and returns the set of entities that reach the target, together with the groups that contain any impacted leaf. Figure 7 illustrates the idea: modifying `User.java` marks every entity that transitively depends on it, and their containing groups, as impacted.

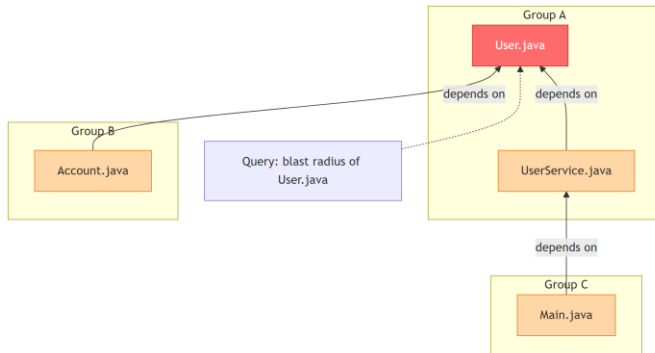


Figure 7. Blast-radius as reverse reachability: the target (red) and all its transitive dependents (amber) are returned, along with the groups that contain them.

The traversal uses a visited set so that cyclic dependency graphs terminate and each node appears at most once; it is deterministic, returning an identical set on repeated queries over an unchanged hierarchy. A node with no incoming dependencies yields a blast radius containing only itself; an unknown or empty node identifier returns a typed error and leaves the hierarchy unchanged. We are explicit about the limits of this analysis. The blast radius is static reachability: it captures dependencies visible in the parsed structure. It does not capture dynamic dependencies - reflection, dependency injection, and string-based lookups, all common in Java and Spring - and will therefore under-count impact in codebases that rely heavily on those mechanisms. We report blast radius as a useful navigational and impact-scoping aid, not as a sound over-approximation of all runtime effects.

IX. Implementation

RepoHIVE is implemented in TypeScript on Node.js, organized as an npm-workspaces monorepo. This choice aligns the grouping engine with the React/React Flow viewer (a single language across the stack) and gives access to a mature graph ecosystem. The core libraries are: graphology for the in-memory directed weighted graph and its traversal

utilities; graphology-communities-louvain as the Phase-1 community-detection backend behind the CommunityDetector interface, seeded deterministically; graphology-metrics for the optional Newman modularity signal; and fast-check for property-based testing. Tree-Sitter's Java grammar is the only language binding required for Phase 1.

The system is deliberately scoped to Java first. Java is the most-studied language in the software-clustering literature [10], its explicit imports make cross-file dependency resolution tractable relative to dynamically typed languages, and Tree-Sitter's Java grammar is mature. The data model and interfaces are kept general, so additional ecosystems (Node, Python, monorepos) and additional region strategies (module and directory subtrees) can be added later without interface changes. Storage is plain JSON files on disk. At the thousands-of-files scale this project targets, no database is required; a graph-native store remains a documented future option behind the storage seam, to be considered only if a hosted, multi-user service is built.

X. Evaluation Design

The evaluation is framed strictly around code navigation - helping a developer or agent move through and understand an unfamiliar codebase - and explicitly not around recovering a ground-truth architecture. We make no claim that reconstructed groups correspond to intended modules; the only question is whether the resulting hierarchy is easier to navigate. All three baseline configurations run the identical pipeline and differ only in the construction decision, so any difference is attributable to the construction policy alone: Always-preserve forces a preserve action for every region (equivalently, set the boundary below every score); Always-reconstruct forces a reconstruct action for every region (boundary above every score); Adaptive is the per-region decision driven by the structural quality score and the calibrated boundary. Because the baselines are reachable purely through configuration, the comparison is clean, deterministic, and requires no separate code path.

Navigation-oriented metrics, each computed deterministically from the produced hierarchy:

- Expansion steps to locate a file - the number of group levels that must be expanded to reach a file from the root, reported as a distribution over all files.
- Average and maximum branching factor - children per group, and how evenly branching is distributed (a hierarchy that respects the size bound but is lopsided is harder to scan).
- Cross-group edge count and weight per level - fewer and lighter cross-group edges mean a navigator following a dependency more often stays within a group, indicating groups that localize related code.

- Balance of sibling group sizes - the spread of group sizes at each level.
- Stability across input permutations - the navigation metrics must be identical when the input node/edge order is permuted; a navigation aid whose shape changes under reordering is not trustworthy.

Sensitivity analysis. Because every run records each region's score and the boundary, a sweep over the boundary (and, secondarily, the per-metric weights and the cohesion constant) is a pure post-processing exercise: for each boundary value the recorded scores reproduce the decisions, and the navigation metrics can be recomputed without re-running assessment. Plotting the navigation metrics against the boundary shows how sensitive navigability is to where the preserve/reconstruct line is drawn, and the decision-confidence distribution identifies the regions that flip near it. This is what turns the boundary into an empirically calibrated, auditable parameter and forms the core of the research defense - again, strictly in service of navigation, not architecture reconstruction.

The intended evaluation corpus is a set of open-source Java repositories at the thousands-of-files scale, consistent with the problem statement and with the Java-based benchmark systems standard in the clustering literature [10].

XI. Preliminary Results

At the time of writing, the parsing layer and the JSON contract are implemented and validated, and the grouping layer is in active development. We report the parser results as preliminary evidence that the front of the pipeline works end to end and produces a contract-conformant artifact at realistic scale.

Parser validation. The parser was exercised on two fixtures. A hand-written multi-package Java fixture - with nested and inner types, overloaded methods, and cross-file imports - produces a graph of 29 nodes and 5 edges, small enough that the expected output can be checked by hand and the determinism digest kept stable. A real third-party Spring Boot application ("Vantage"), comprising 158 Java source files, produces a graph of 803 nodes and 128 edges. The parser is covered by 102 automated tests, including property-based tests for determinism (identical SHA-256 digest of graph.json across repeated runs), contract conformance, edge uniqueness, and the absence of self-edges and function-to-function edges that the contract disallows.

Determinism, measured. For the parser, repeated runs over the same repository produce a byte-identical graph.json (verified by an identical SHA-256 digest across three runs). This is a measured determinism result for the implemented stage, distinct from the estimates below.

Performance (engineering estimates, not measured benchmarks). We label the following as estimates because they have not yet been measured on a full-scale corpus. For a target repository of roughly 4,000 Java files and about 50,000 edges we estimate: parsing on the order of tens of seconds to a couple of minutes, dominated by disk I/O in reading thousands of files rather than by parsing itself; grouping on the order of seconds to about a minute, since Louvain handles tens of thousands of edges comfortably and runs only on the reconstruct regions; and a graph.json on the order of 10–20 MB, because it stores identifiers and small integers rather than source text. We expect the full pipeline to be a one-time batch job of a few minutes, not a repeated interactive wait. These figures are stated to set expectations and will be replaced by measured numbers as the grouping layer completes; we deliberately avoid headline multipliers or "orders-of-magnitude" claims until they are measured.

XII. Limitations and Threats to Validity

We state the limitations openly, because they are what a careful reader will probe.

- Adaptive superiority is a hypothesis, not a proven result. Claim B - that adaptive construction beats both global baselines on mixed-quality repositories - is the subject of the planned evaluation (Section X), not an established finding. The honest expected outcome is graceful degradation: adaptive should never be worse than the better global baseline, and strictly better only where region quality is genuinely mixed.
- Blast radius is static. As noted in Section VIII, it misses reflection, dependency injection, and string-based lookups, and will under-count impact in Java/Spring code that relies on them.
- Cohesion is scale-relative in Phase 1. Cohesion is normalized by node count, so two regions with the same internal-strength-per-node ratio but very different sizes are treated as equivalent; a density-based normalization (by the number of possible intra-region edges) is a documented future refinement.
- Frequency signals start simple. In the current parser, method-call frequency and shared-type count are not yet fully populated, so the initial dependency strength is import-driven; the contract shape is correct from day one, and the signals will be sharpened without breaking it.
- Single language and single scale tier. Results will be demonstrated at the thousands-of-files scale on Java; we do not claim scaling to hundreds of thousands of files or to arbitrary languages, both of which are explicitly future work.
- Token-saving claims are bounded. A graph-over-raw-files saving is shared with existing tools and is table stakes; our specific claim is only the incremental saving from

delivering a hierarchical branch rather than a flat graph, which we will measure rather than assert.

- We do not claim overall superiority over mature platforms. Broad code-intelligence systems address a far wider feature set; our contribution is narrow and specific - deterministic, adaptive, hierarchical navigation - and we make no claim beyond that slice.

XIII. Conclusion and Future Work

We presented RepoHIVE, a system that turns the flat dependency graph of a large Java repository into a deterministic, navigable, multi-level hierarchy. Its distinguishing idea is adaptive per-region construction: measure each region's structural quality and preserve it when it is already well-structured or reconstruct it via community detection when it is not, rather than imposing one global grouping method. Every score, decision, and parameter is recorded, making the result reproducible and the decision boundary an auditable, tunable research parameter. A stable JSON contract lets one engine serve a human semantic-zoom viewer, a bounded-context AI-agent consumer, and a static blast-radius query. The parsing layer is implemented and validated at realistic scale; the grouping layer and the navigation-oriented evaluation are the immediate next steps.

Future work follows the seams the architecture already exposes. The CommunityDetector interface makes a Louvain-versus-Leiden comparison a low-cost experiment. Co-change ("logical") dependencies from version history [5] can enrich the edge signal behind the same contract. A density-based cohesion measure addresses the scale-relative caveat. Beyond the engine, distribution wrappers (a CLI, an editor extension, a query server) and an optional graph-native store attach through the JSON and storage seams without engine rework, and a semantic layer built on embeddings could support search and cluster naming - never grouping, which remains deterministic and structural by design.

References

- [1] R. Ehsani, E. Parra, S. Haiduc, and P. Chatterjee, "Hierarchical knowledge injection for improving LLM-based program repair," arXiv:2506.24015, 2025.
- [2] M. Athale and V. Vaddina, "Knowledge graph based repository-level code generation," arXiv:2505.14394, 2025.
- [3] N. Dhulshette, S. Shah, and V. Kulkarni, "Hierarchical repository-level code summarization for business applications using local LLMs," arXiv:2501.07857, 2025.
- [4] S. Druskat et al., "Better architecture, better software, better research," Computing in Science & Engineering, vol. 27, no. 2, pp. 45–57, 2025, doi: 10.1109/MCSE.2025.3573887.
- [5] A. D. Stana and I. Şora, "Refining software clustering: The impact of code co-changes on architectural reconstruction," IEEE Access, vol. 13, pp. 132126–132145, 2025, doi: 10.1109/ACCESS.2025.3592777.
- [6] D. Sounthiraraj, J. Hancock, Y. Kortam, A. Javvaji, P. Singh, and S. Shankar, "Code-Craft: Hierarchical graph-based code summarization for enhanced context retrieval," arXiv:2504.08975, 2025.
- [7] S. Ouyang et al., "RepoGraph: Enhancing AI software engineering with repository-level code graph," in Proc. Int. Conf. on Learning Representations (ICLR), 2025, arXiv:2410.14684.
- [8] N. F. Liu et al., "Lost in the middle: How language models use long contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157–173, 2024, arXiv:2307.03172.
- [9] F. Zhang et al., "RepoCoder: Repository-level code completion through iterative retrieval and generation," in Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2023, arXiv:2303.12570.
- [10] Q. I. Sarhan, B. S. Ahmed, M. Bures, and K. Z. Zamli, "Software module clustering: An in-depth literature analysis," IEEE Transactions on Software Engineering, vol. 48, no. 6, pp. 1905–1928, 2022, doi: 10.1109/TSE.2020.3042553.